

第14章 类型系统进阶

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已完成前13章;熟悉 C 的类型、NULL、typedef、union 与 enum

🎯 本章学习目标

- 掌握类型推导的边界:什么时候编译器能猜、什么时候必须显式标注
- 掌握 Option 的完整用法,理解 空安全:可选性进入类型系统,消灭 NULL 事故
- 掌握 enum 和类型:用类型表达"选项集合",与 C 的 enum+union+tag 对照
- 掌握 newtype 模式:用类型包装消灭单位混淆与"类型即文档"
- 深化值语义 vs 引用语义:Copy/Clone/借用三态在生产代码里的完整图景
- 理解类型系统与 trait 的交汇:impl Trait 返回、?Sized、类型别名
- 用领域建模案例(订单系统)把全书知识串成完整闭环
- 获得"进入生产后不再有语法困惑"的最后一块拼图

💡 从 C 到 Rust:本章主题如何迁移

全书最后一章,回答一个总问题:C 程序员的类型直觉,在 Rust 里哪里需要升级?答案是三处:**第一,可选性**——C 的 NULL 是"值层面的哨兵",谁都能传、传了才知道错;Rust 的 Option<T> 是"类型层面的事实",函数签名直接声明"这里可能没有值",编译器强迫你处理。**第二,区分度**——C 的 typedef 只是别名,int 换了个名字还是 int,米和秒照样能相加;Rust 的 newtype 让"米"和"秒"成为两个互不相通的类型,单位错误在编译期暴露。**第三,值 vs 引用**——C 只有"值/指针"两态,指针传参的意图(只读/修改/转移)全靠注释;Rust 的值/借用/移动三态(第4章)是类型系统的一部分,函数签名即意图。

这三处升级的共同本质:把"约定"变成"类型"。C 靠注释和纪律维持的约定,在 Rust 里由编译器强制执行。本章前三节讲 Option 与 enum(建模可选与选项),§14.4 讲 newtype(建模区分),§14.5 讲值/引用语义(建模所有权),§14.6 讲与 trait 的交汇,最后 §14.7 用订单领域模型把全书 14 章串成一张生产地图。

📦 前置知识自查

- C 的 NULL、typedef、union、enum、指针传参的三种意图(只读/修改/转移)
- 第1章的 let 与类型标注、第2章的运算符、第3章的 match/if let
- 第4章的参数传递三态、第5章的所有权/借用、第6章的 trait
- 第9章的 Result 与 ?(本章 Option 与 Result 对照)
- 第13章的模块与 crate(本章示例使用库结构)

本章学习路线

1. **第一阶段(约1小时):**§14.1–§14.2,类型推导边界与 Option——把"可能没有"变成类型事实
2. **第二阶段(约1.5小时):**§14.3–§14.4,enum 建模与 newtype——领域建模的两大武器
3. **第三阶段(约1.5小时):**§14.5–§14.6,值/引用语义与 trait 交汇——类型系统的完整图景
4. **第四阶段(约1小时):**§14.7,订单领域模型实验与全书收官——把 14 章串成闭环
5. **验收标准:**给你一个业务需求,能用"enum 建模状态、newtype 建模单位、Option 建模可选、trait 建模行为"给出完整设计

本章知识导图

- §14.1 类型推导:边界与默认规则
- §14.2 Option:空安全 vs C 的 NULL
- §14.3 enum 和类型:状态与选项的建模
- §14.4 newtype 模式:类型即文档
- §14.5 值语义 vs 引用语义:深化
- §14.6 类型系统与 trait 的交汇
- §14.7 收官:订单领域模型与全书地图
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§14.1 类型推导:边界与默认规则

14.1.1 哪里能推导,哪里必须显式

```
fn main() {  
    // 能推导:let 绑定、函数调用结果  
    let x = 42;                // i32(整数默认)  
    let y = 3.14;              // f64(浮点默认)  
    let s = String::from("hi"); // 构造器决定类型  
  
    // 必须显式:函数签名(类型是契约的一部分)  
    // fn f(x) { ... }          // ERROR: 必须写 fn f(x: i32)  
  
    // 必须显式:泛型函数的类型参数(编译器不知道你要哪个)  
    let n: u32 = "42".parse().expect("字面量必然可解析"); // 类型标注决定 parse 的目标  
    // "42".parse();          // ERROR: 无法推断  
  
    // 必须显式:集合元素的类型(空集合无法从内容推断)  
    let empty: Vec<i32> = Vec::new();  
    let m: std::collections::HashMap<u32, String> = Default::default();  
    println!("{n} {empty:?} {:?}", m.len());  
}
```

14.1.2 推导的边界:不跨越函数与闭包体

```
// 推导是"局部"的:一个函数内的类型问题,编译器不会到另一个函数里找答案
fn identity<T>(x: T) -> T { x }

fn main() {
    // 泛型调用需要"推断点":赋值目标或参数类型
    let a: u64 = identity(5);           // T = u64
    let b = identity(5u8);             // 字面量后缀给出 T
    println!("{a} {b}");

    // 闭包参数:能推导时省略标注;需要时显式
    let double = |x| x * 2;            // 从使用处推导
    // let trouble = |x| { 复杂到推断不出 }; // 罕见;显式标注参数即可
    let f = |x: f64| x * 2.0;          // 显式标注参数类型
    println!("{}", double(21), f(1.5));
}
```

⚠ 易错点 1:整数默认 i32,浮点默认 f64——循环与索引的坑

let n = 0; 是 i32;数组长度是 usize。索引/取长度相关的推导冲突(IndexError: expected usize)是 C 转 Rust 高频报错——C 里 int 到处通用,Rust 里 usize 与 i32 不互转(第2章零隐式转换)。对策:循环计数与集合下标统一用 usize;需要数学运算时再显式转换 as i64 等。

§14.2 Option:空安全 vs C 的 NULL

14.2.1 NULL 的灾难与 Option 的答案

```
// C:NULL 是"值层面的哨兵",函数签名看不出会不会返回 NULL
// const char *find_name(int id);    // 可能返回 NULL?看文档才知道
// char *p = find_name(42);
// strcpy(p, "x");                  // p 是 NULL?崩溃(或者更糟:静默)

// Rust:可选性写在签名里,编译器强迫处理
fn find_name(id: u32) -> Option<String> {    // 明确:可能没有
    if id == 42 {
        Some(String::from("小明"))
    } else {
        None                                // 显式表示"没有"
    }
}

fn main() {
    let name = find_name(7);
    // 三种处理方式:
    // 1) match 穷尽(第3章)
    match &name {
        Some(n) => println!("找到:{n}"),
        None => println!("未找到"),
    }
    // 2) 链式处理:map 只对 Some 生效
    let len = name.as_ref().map(|n| n.len()).unwrap_or(0);
    println!("长度:{len}");
    // 3) 不存在的值不可能"被当作存在"使用——类型检查兜底
}
```

14.2.2 Option 的常用组合拳

```
fn main() {
    let a: Option<i32> = Some(10);
    let b: Option<i32> = None;

    // map:Some 则变换,None 保持 None
    println!("{:?}", a.map(|x| x * 2));      // Some(20)
    println!("{:?}", b.map(|x| x * 2));      // None

    // and_then:变换结果也是 Option(展平),对照 C 的"返回值再判空"
    println!("{:?}", a.and_then(|x| if x > 5 { Some(x - 5) } else { None })); // Some(5)

    // unwrap_or / unwrap_or_else:兜底值(延迟计算用闭包)
    println!("{}", b.unwrap_or(0));          // 0
    println!("{}", a.unwrap_or_else(|| 99)); // 10

    // ? 运算符:Option 与 Result 可以互转(第9章 Result)
    // fn f() -> Option<i32> { let x = a?; Some(x + 1) }

    // 判空/取引用
    println!("is_some:{}, is_none:{}, a.is_some(), b.is_none());
    println!("{:?}", a.as_ref());            // Some(&10):借用不移动
}
```

C 的空值处理	Rust Option
NULL 指针 / 哨兵值(-1、EOF)	Option<T>:Some(T) / None,类型级表达
返回值是否可能为空:看文档/注释	签名自带:Option<T> 就是"可能没有"
忘记判空:运行期崩溃或 UB	编译期强迫处理:None 无法当作 T 用
判空后层层 if 缩进	map / and_then / ? 链式,无嵌套
空串/0 作为"伪空"的妥协	None 是唯一空值,语义精确

§14.3 enum 和类型:状态与选项的建模

14.3.1 用类型表达"选项集合"

```
// C 的 enum + union + tag 三件套 → Rust 一个 enum 搞定
// C:
//   enum OrderStatus { PENDING, PAID, SHIPPED, CANCELLED };
//   // 状态值只是数字;switch 忘了处理新状态?编译器不检查

// Rust:enum 是完整的类型,携带数据,穷尽检查
enum OrderStatus {
    Pending,                                // 无数据
    Paid { amount: f64, time: String },    // 带数据的变体(对照 union)
    Shipped(String),                        // 带单个数据:运单号
    Cancelled(String),                      // 原因
}

fn describe(status: &OrderStatus) -> String {
    match status {
        OrderStatus::Pending => "等待支付".to_string(),
        OrderStatus::Paid { amount, .. } => format!("已支付 {amount} 元"),
        OrderStatus::Shipped(no) => format!("已发货,运单 {no}"),
        OrderStatus::Cancelled(reason) => format!("已取消:{reason}"),
        // match 穷尽:以后新增变体,这里编译错误,逼你处理
    }
}

fn main() {
    let s = OrderStatus::Paid { amount: 99.0, time: "10:00".into() };
    println!("{}", describe(&s));
}
```

14.3.2 enum 作为"选项集"的实战形态

```
// 生产高频模式:enum 表示"几种可能的形态",数据随变体走
enum Config {
    Port(u16),                // 单个值
    Addr { host: String, port: u16 }, // 命名结构
}

impl Config {
    fn display(&self) -> String {
        match self {
            Config::Port(p) => format!("只配端口:{p}"),
            Config::Addr { host, port } => format!("{host}:{port}"),
        }
    }
}

fn main() {
    let c1 = Config::Port(8080);
    let c2 = Config::Addr { host: "0.0.0.0".into(), port: 9090 };
    println!("{}", c1.display(), c2.display());
    // C 对照:union + kind 字段 + switch;Rust 把三样收进一个类型,
    // 且"哪个变体有哪些字段"由编译器检查—取错字段直接编译错误
}
```

⚠ 易错点 2:enum 里塞"状态数据" vs 用字段表达

状态机建模的常见分歧:状态相关的数据放 enum 变体里(如上例),与状态无关的数据放 struct 字段里。判据:该数据是否只在某状态下存在?是→放变体;否→放结构体字段。把"所有状态共用的数据"塞进变体,会让 match 每个分支重复处理,还容易漏。生产直觉:先画状态图,再定 enum,最后定 struct 字段。

§14.4 newtype 模式:类型即文档

14.4.1 typedef 的局限与 newtype 的升级

```
// C:typedef 只是别名,米和秒都是 double,照样能相加
// typedef double meters;
// typedef double seconds;
// meters m = 100.0; seconds s = 5.0;
// double v = m / s;    // 合法但..... m/s 是速度,单位混淆编译器不管

// Rust:newtype—包一层,单位成为独立类型
struct Meters(f64);
struct Seconds(f64);

impl Meters {
    fn new(v: f64) -> Meters { Meters(v) }
    fn value(&self) -> f64 { self.0 }
}

fn speed(distance: Meters, time: Seconds) -> f64 {
    distance.value() / time.value()    // 取内部值需显式方法
}

fn main() {
    let d = Meters::new(100.0);
    let t = Seconds::new(5.0);
    println!("速度:{}", speed(d, t));
    // speed(t, d);          // ERROR:类型不匹配—单位错了,编译期报错!
    // let wrong = d + t;    // ERROR:Meters 与 Seconds 不能相加(无 Add impl)
}
```


14.4.2 newtype 的三个生产用途

```
// 用途一:单位/概念区分(上例)
// 用途二:为外部类型附加本 crate 的方法(孤儿规则限制的解药)
use std::collections::HashMap;

struct UserMap(HashMap<u32, String>);    // 包装外部类型

impl UserMap {
    fn new() -> UserMap { UserMap(HashMap::new()) }
    fn insert(&mut self, id: u32, name: &str) {
        self.0.insert(id, name.to_string());
    }
    fn total(&self) -> usize { self.0.len() }
}

// 用途三:给"同一底层类型"的不同角色加保护
struct RawText(String);    // 未净化的文本
struct Sanitized(String);  // 已净化的文本
// 只有 Sanitized 才能进渲染函数 → 数据流的安全阶段在类型里可见

fn main() {
    let mut users = UserMap::new();
    users.insert(1, "张三");
    println!("用户数:{}", users.total());

    let raw = RawText("<script>alert(1)</script>".to_string());
    // render(&raw);    // ERROR:RawText 不是 Sanitized—类型防线
    let _safe = Sanitized(raw.0);    // 显式净化转换
    println!("净化管线类型安全");
}
```

C	Rust
typedef double meters;只是别名	struct Meters(f64);独立类型,互不兼容
单位混淆:编译通过,运行期算错	单位混淆:编译错误
int 的"多个角色"靠命名约定	newtype 让角色成为类型
外部类型不能加方法	newtype 包装后可为包装类型实现 trait
数据流安全靠纪律	RawText/Sanitized 式管道,编译器把关

§14.5 值语义 vs 引用语义:深化

14.5.1 三条语义轴

```
// 轴一:复制(Copy)vs 移动(Move)
// 数字/字符/小值:Copy,赋值即拷贝
let a = 5i32;
let b = a; // 拷贝:b 与 a 都是 5,独立
// 大值/String/Vec:Move,赋值转移所有权
let s = String::from("hi");
let t = s; // 移动:s 失效
// println!("{}", s); // ERROR: use of moved value

// 轴二:共享借用(&T,只读)vs 独占借用(&mut T,可写)
// 规则(第5章):不可变借用可无数;可变借用唯一
// 对应 C:const T* 与 T*—但 Rust 的规则是编译器强制的

// 轴三:所有权(拥有/释放)vs 借用(使用不拥有)
// Box/String/Vec:拥有; &T:借用;生命周期由编译器管

#[derive(Debug, Clone)]
struct Big { data: Vec<u32> } // 大类型:Move + 显式 clone

fn main() {
    let mut big = Big { data: vec![1, 2, 3] };
    let copy_of_big = big.clone(); // 显式深拷贝(对照 memcpy)
    let borrowed = &big; // 共享借用:可同时多个
    println!("{:?} {:?}", borrowed, copy_of_big);

    let exclusive = &mut big; // 独占借用
    exclusive.data.push(4);
    println!("{:?}", big); // 借用结束,回到拥有者
}
```

14.5.2 从 C 的指针三意图到 Rust 的三态

C 的传参意图	C 写法	Rust 三态
只读查看	const T* p	&T(共享借用)
就地修改	T* p	&mut T(独占借用)
转移所有权	T* p + 约定"调用方负责 free"	T(按值传入,所有权转移)
复制一份	memcpy / 手写	T: Clone 的 .clone() / T: Copy

★ 重点:C 的意图靠注释,Rust 的意图是类型

读 C 代码时,你要从 `void f(T* p, const T* q)` 里猜哪个是读、哪个是写、哪个是转移——猜错的代价是悬垂与泄漏。Rust 的函数签名直接回答这三个问题: `f(&T)` 不拥有、`f(&mut T)` 独占改写、`f(T)` 拿走所有权。这三态(第4章)与所有权(第5章)是全书的心跳;本章的深化在于:把三态放进"值语义 vs 引用语义"的完整坐标里——Copy 类型天然值语义,C 的"struct 按值传"在这里得到类型化;引用语义类型(字符串、容器、流)的共享与独占由借用规则管。

§14.6 类型系统与 trait 的交汇

14.6.1 impl Trait 返回:隐藏类型,保留契约

```
// 返回"实现了某 trait 的类型",但不暴露具体类型
fn make_counter() -> impl Iterator<Item = u32> {
    (1..=5) // 具体类型是 RangeInclusive,调用方不需要知道
}

fn main() {
    let total: u32 = make_counter().sum();
    println!("总和:{total}");

    // 与泛型的对照:返回位置 impl Trait 是"匿名具体类型";
    // 调用方只能按 trait 用,不能做 trait 之外的操作——契约即边界
}
```

14.6.2 Sized 与 ?Sized、类型别名

```
// 默认 T: Sized(大小编译期已知);dyn Trait 与 [T] 是不定大小
fn takes_sized<T>(_x: T) {} // T 必须有确定大小(默认)

// ?Sized:允许不定大小(如 &dyn Trait 的借用)
fn accepts_any<T: ?Sized>(_x: &T) {} // &str / &dyn Trait 都行

// 类型别名:与 C typedef 相同(纯别名,不提供 newtype 的安全性)
type Price = f64; // 别名:Price 就是 f64
type Result2<T> = std::result::Result<T, String>; // 简化签名

fn main() {
    let p: Price = 99.9; // 与 f64 完全互通
    println!("{}", p);
    let r: Result2<u32> = Ok(1);
    println!("{}", r);

    // 别名的取舍:C 的 typedef 习惯在 Rust 里的正确位置:
    // 需要"简洁"用 type;需要"区分与安全"用 newtype(§14.4)
}
```

14.6.3 类型系统全景:三股力量

力量	工具	解决的问题
可选性	Option / Result	"可能没有"与"可能失败"进入类型
区分度	newtype / enum / 类型参数	单位、角色、状态不再混淆
行为契约	trait / trait bound / dyn	"能做什么"由类型声明

§14.7 收官:订单领域模型与全书地图

14.7.1 全书 14 章的知识闭环

章节	一句话贡献
1 数据类型与变量	let 绑定、强类型、类型系统带着所有权
2 运算符与表达式	无隐式转换、无 ++、表达式风格
3 流程控制	match 穷尽、if let、loop 带值
4 函数与方法	值/借用/移动三态、闭包三态
5 内存管理	所有权三法则、生命周期、智能指针
6 面向对象与 trait	行为契约、组合优于继承、两种分派
7 文件与 IO	Result 世界、路径类型、serde
8 网络通信	TCP/UDP、字节流定界、HTTP
9 并发与异常	错误三件套、Send/Sync、通道、tokio
10 泛型与元编程	单态化、trait bound、宏
11 反射与动态特性	无反射;Any/宏的编译期替代
12 注解与编译期配置	属性系统、#[cfg]、features
13 模块与 Cargo 工程	模块树、Cargo 全流程、workspace
14 类型系统进阶	Option、enum、newtype、语义三轴(本章)

14.7.2 进入生产后的三条心法

心法一:类型先行

写业务前先定义类型:状态用 enum、可选用 Option、单位用 newtype、行为用 trait——类型定义得越精确,后续 match/分支越少,编译器帮你查的错误越多。C 程序员最划算的习惯升级:写函数前先写签名,签名即文档,签名即测试。

心法二:编译错误是老师的答案

Rust 编译器报错时,报错信息通常指向"缺哪个 trait、哪个生命周期、哪个所有权转移"。生产里 90% 的修复路径:读错误 → 补约束/换借用 → 过。不要与编译器搏斗,顺着它的引导走。

🚀 心法三:unsafe 是边界,不是习惯

FFI、裸指针、极致性能才碰 unsafe,并用 unsafe 块包裹、注释说明不变量(第5章)。Rust 的安全保证是生产级代码的护城河——护城河外的操作,要像 C 一样小心,但范围要缩到最小。

⚠️ 易错点 3:newtype 不是越多越好

newtype 的安全来自"转换要显式",但每个 newtype 都增加一层 .0/方法的样板。判据:两个概念在业务上是否真的"混用即事故"?物理量、货币、ID、已净化文本——是,值得包;同义别名(两个都只是"一段文本")——用 type 别名或直接用 String。过度包装会让代码像裹了多层保鲜膜,可读性被样板反噬。生产经验:先以 type 别名起步,出现"传错角色"的真实事故记录后再升级为 newtype,让类型跟随痛苦而生长。

本章速查表

主题	结论 / 写法
类型推导	let 绑定/调用结果可推导;函数签名、泛型参数、空集合必须显式
整数/浮点默认	i32 / f64;索引与长度用 usize(与 i32 不隐式互转)
Option 形态	Some(T) / None;签名自带"可能没有"
Option 处理	match / if let / map / and_then / unwrap_or / ?
enum 建模	变体可带数据:enum S { A, B {x: f64}, C(String) }
穷尽检查	match 必须覆盖全部变体;新增变体编译报错逼你处理
newtype	struct Meters(f64);互不兼容的类型;单位/角色/外部类型包装
type 别名	type Price = f64;纯别名,无安全性(与 typedef 相同)
语义三轴	Copy/Move;共享借用 &T;独占借用 &mut T;所有权/借用
impl Trait 返回	fn f() -> impl Iterator<Item = u32>;隐藏类型保留契约
?Sized	<T: ?Sized> 允许不定大小(如 &dyn Trait)
类型系统三力	可选性(Option/Result)+ 区分度(newtype/enum)+ 契约(trait)

最小必会清单

- 能说出类型推导的边界(签名/泛型/空集合必须显式)与默认类型规则
- 能用 Option 建模"可能没有",并说出与 C NULL 的本质差别
- 能写出带数据的 enum,并利用 match 穷尽检查获得编译期保障
- 能用 newtype 建模单位/角色,说出与 typedef 的本质差别
- 能说清 Copy/Clone/Move/借用之间的关系与各自场景
- 能从函数签名读出"只读/改写/转移"三态(第4章能力的最终形态)
- 会用 impl Trait 返回隐藏具体类型
- 能区分 type 别名与 newtype 的适用场景
- 能用"类型先行"的方法为业务建模(enum 状态 + newtype 单位 + Option 可选)

- 能画出全书 14 章的知识地图,并说出每章在生产中的角色

自测题

Q 1. 为什么函数参数必须写类型,而 let 绑定可以不写?这对 C 程序员意味着什么?

✓ 参考答案

函数签名是跨调用方的契约:类型是接口的一部分,编译器要据此检查所有调用点;let 绑定是局部事实,可从初始化表达式推导。对 C 程序员:这比 C 更严格也更安全——C 的函数声明(头文件)里类型不齐是常态,靠调用方猜;Rust 的签名即全部契约,猜的部分由编译器完成。

Q 2. Option<T> 与 C 的"返回 NULL / 哨兵值"相比,消除了哪几类 bug?

✓ 参考答案

三类:① 忘记判空——None 无法被当作 T 使用,编译期强制处理;② 语义歧义——C 里 0/-1/空串都当过"空",Option 只有 None 一个空值;③ 误判生命周期——C 的返回指针可能是悬垂(指向已释放栈),Option 的值语义/借用规则在类型层排除。剩下的"选择",用 map/? 链式表达,不产生嵌套缩进。

Q 3. C 的 enum + union + tag 与 Rust 的 enum,编译期保障差在哪里?

✓ 参考答案

C:enum 是数字集合,switch 漏掉新分支不报错(或靠 -Wswitch 部分检查);union 字段与 tag 的对应靠程序员约定,取错字段是 UB。Rust:enum 是代数数据类型,变体与携带数据一一对应,取某变体的数据必须先匹配该变体(编译器检查);match 穷尽性保证新增变体时所有分支都被审计——"状态机漏分支"从运行期灾难变成编译期必报。

Q 4. typedef double meters 与 struct Meters(f64) 在生产上差出什么?

✓ 参考答案

typedef 的 meters 仍是 double:米与秒、价格与数量可以互相赋值、相加,单位错误全部留到运行期(数据算错,最危险的那种 bug)。newtype 的 Meters 是与 f64 互不兼容的独立类型:传错单位编译失败;需要取值必须显式 .0/方法——每个单位转换在代码里可见。生产收益:涉及物理量、货币、ID 的系统,newtype 把一整类事故消灭在编译期。

Q 5. 从函数签名 &T / &mut T / T 分别读出的契约是什么?C 里对应的注释长什么样?

✓ 参考答案

&T:只读共享,调用方保留所有权,可能同时被多处读;&mut T:独占改写,调用期间别无借用;T:所有权转移,调用方不再拥有。C 的对应:const T* / T* / "T* 且约定调用方负责 free"。差别:前两者编译器强制(违反借用规则编译失败),转移在 C 里只是口头约定——所以 C 的悬垂与 double-free 频发,而 Rust 的签名即保证。

Q 6. 一个电商系统里,订单金额、商品 ID、用户 ID 都用 u64 表示,会出什么问题?如何修复?

✅ 参考答案

问题:三者的 u64 互相兼容,传参错位(把商品 ID 当金额、把用户 ID 当商品 ID)编译通过、运行期产生错误业务数据——这是"类型区分度"缺失。修复:newtype 建模:struct Money(u64); struct ProductId(u64); struct UserId(u64);各配实现。从此金额函数只收 Money,ID 函数只收对应 ID,错位在编译期报错。配套:enum 建模订单状态、Option 建模"优惠券可能没有",这就是"类型先行"的完整生产形态。

章末实验题:订单领域模型

📁 实验 14:订单领域模型

实验目标:用本章全部武器建模一个订单系统:newtype 建模金额与 ID、enum 建模订单状态与支付方式、Option 建模可选信息、trait 建模行为;并写一个命令行演示全流程。

实验要求:

- 任务 1:newtype:Money、ProductId、UserId(各自实现 Display)(覆盖:newtype §14.4)
- 任务 2:enum OrderStatus 与 PaymentMethod(带数据变体)(覆盖:enum §14.3)
- 任务 3:struct Order:用户、商品列表、金额、状态、可选优惠券(覆盖:Option §14.2)
- 任务 4:impl 方法:total(汇总金额)、apply_coupon(返回 Result 或 Option)(覆盖:impl 与错误 §9/§14.2)
- 任务 5:trait Billable:Order 与 Invoice 都实现"应付款"行为(覆盖:trait §6)
- 任务 6:main 演示:建订单 → 加优惠券 → 支付 → 状态流转,全程类型安全(覆盖:收官 §14.7)
- 加分:尝试把 Order::total 的金额类型改成 u64 直接相加,观察编译错误(单位防线)

第一步 **一步:**cargo new order-demo;先写三个 newtype 与 Display

第二步 **二步:**写两个 enum(状态与支付方式),为状态写描述方法

第三步 **三步:**写 Order 结构与方法(字段含 Option<Coupon>)

第四步 **四步:**写 Billable trait,Order 与 Invoice 各自实现

第五步 **五步:**main 里完整走一遍流程,编译通过后做加分项(去掉 Money 看编译错)

✅ 参考答案要点

```
// ===== 实验 14:订单领域模型 =====
// 覆盖知识点:newtype 建模(§14.4)、enum 状态建模(§14.3)、
//          Option 可选性(§14.2)、trait 行为契约(§6)、
//          impl 方法与错误处理(§9)、收官串联(§14.7)

use std::fmt;

// ---- 任务 1:newtype 三件套:单位与角色独立 ----
struct Money(u64);           // 金额:分
struct ProductId(u64);
struct UserId(u64);

impl fmt::Display for Money {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{:.2} 元", self.0 as f64 / 100.0)
```

```

    }
}

impl fmt::Display for ProductId {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "商品#{}", self.0)
    }
}

impl fmt::Display for UserId {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "用户#{}", self.0)
    }
}

impl Money {
    fn add(&self, other: &Money) -> Money { Money(self.0 + other.0) }
    fn is_zero(&self) -> bool { self.0 == 0 }
}

// ---- 任务 2:enum 状态与支付方式 ----
enum OrderStatus {
    Pending, // 待支付
    Paid { method: PaymentMethod, at: String }, // 已支付(带数据)
    Shipped(String), // 已发货(运单号)
    Cancelled(String), // 已取消(原因)
}

enum PaymentMethod {
    WechatPay,
    Alipay,
    BankCard(String), // 尾号
}

impl OrderStatus {
    fn describe(&self) -> String {
        match self {
            OrderStatus::Pending => "待支付".to_string(),
            OrderStatus::Paid { method, .. } => {
                format!("已支付({})", match method {
                    PaymentMethod::WechatPay => "微信支付",
                    PaymentMethod::Alipay => "支付宝",
                    PaymentMethod::BankCard(last4) => {
                        return format!("已支付(银行卡尾号 {last4})");
                    }
                })
            },
            OrderStatus::Shipped(no) => format!("已发货(运单 {no})"),
            OrderStatus::Cancelled(reason) => format!("已取消:{reason}"),
        }
    }
}

// ---- 任务 2 补充:优惠券(可选信息) ----
struct Coupon {

```



```

    code: String,
    discount: Money,                                // 减免金额
}

// ---- 任务 3:Order 结构 ----
struct Order {
    id: u32,
    user: UserId,
    products: Vec<(ProductId, Money)>,              // 商品与单价
    status: OrderStatus,
    coupon: Option<Coupon>,                          // 可能没有优惠券
}

// ---- 任务 4:订单方法 ----
impl Order {
    fn new(id: u32, user: UserId) -> Order {
        Order {
            id,
            user,
            products: Vec::new(),
            status: OrderStatus::Pending,
            coupon: None,
        }
    }

    fn add_product(&mut self, pid: ProductId, price: Money) {
        self.products.push((pid, price));
    }

    /// 汇总金额:商品总价减去优惠券减免(减不了就报错)
    fn total(&self) -> Result<Money, String> {
        let raw = self.products.iter()
            .fold(Money(0), |acc, (_, p)| acc.add(p));
        match &self.coupon {
            None => Ok(raw),
            Some(c) if c.discount.0 > raw.0 => {
                Err(format!("优惠券 {} 减免超总价", c.code))
            }
            Some(c) => Ok(Money(raw.0 - c.discount.0)),
        }
    }

    /// 支付:仅待支付状态可支付(状态机在类型层把关)
    fn pay(&mut self, method: PaymentMethod) -> Result<(), String> {
        match &self.status {
            OrderStatus::Pending => {
                self.status = OrderStatus::Paid {
                    method,
                    at: "2026-08-26 12:00".to_string(),
                };
                Ok(())
            }
            _ => Err(format!("订单 {} 当前状态不可支付", self.id)),
        }
    }
}

```

```

}

// ---- 任务 5:trait 行为契约:可开票对象 ----
trait Billable {
    fn amount(&self) -> Result<Money, String>;
    fn describe(&self) -> String;
}

impl Billable for Order {
    fn amount(&self) -> Result<Money, String> { self.total() }
    fn describe(&self) -> String {
        format!("订单#{ } { } { }", self.id, self.user, self.status.describe())
    }
}

struct Invoice { no: String, items: Vec<Money> }

impl Billable for Invoice {
    fn amount(&self) -> Result<Money, String> {
        Ok(self.items.iter().fold(Money(0), |a, i| a.add(i)))
    }
    fn describe(&self) -> String {
        format!("发票#{ } 共 { } 项", self.no, self.items.len())
    }
}

fn print_bill(b: &impl Billable) {
    match b.amount() {
        Ok(m) => println!("{ } -> 应付 {m}", b.describe()),
        Err(e) => eprintln!("{ } -> 错误:{e}", b.describe()),
    }
}

// ---- 任务 6:main 全流程演示 ----
fn main() {
    let mut order = Order::new(1, UserId(42));
    order.add_product(ProductId(1001), Money(1999)); // 19.99 元
    order.add_product(ProductId(1002), Money(3000)); // 30 元
    order.coupon = Some(Coupon {
        code: "SAVE5".into(),
        discount: Money(500), // 减 5 元
    });

    print_bill(&order); // 应付 44.99 元
    let _ = order.pay(PaymentMethod::WechatPay);
    print_bill(&order); // 已支付(微信支付)

    // 错误路径:重复支付被拒(状态机)
    match order.pay(PaymentMethod::Alipay) {
        Ok(_) => println!("不该成功"),
        Err(e) => println!("预期错误:{e}"),
    }

    // 发票同样可开票(多态:同一契约,不同实现)
    let invoice = Invoice {

```

```

        no: "INV-001".into(),
        items: vec![Money(1999), Money(500)],
    };
    print_bill(&invoice);

    // 加分演示:直接 u64 相加会被类型拒绝—
    // let bad: u64 = order.total().unwrap().0 + 100; // 需显式 .0,看得见单位
    println!("领域模型演示完毕");
}

```

设计说明:全章的武器在一百行内协同——newtype 保证金额与 ID 不会互换,enum 让订单状态机在类型里显形(match 穷尽保证每个状态都被处理),Option 让"优惠券可能没有"成为签名事实,trait 让订单与发票共用一套"开票"逻辑(第6章组合思维),Result 让"优惠券超抵""状态不可支付"成为显式错误。这就是"类型先行"的完整示范。进阶挑战:给 Money 实现 PartialOrd 与 Add(第6章运算符重载),给 Order 加"只能从未发货状态取消"的状态机守卫——做完这两步,你的 Rust 生产级建模能力就毕业了。

全书完。14 章,从 let 绑定到领域建模,从 malloc/free 到所有权,从 errno 到 Result——愿这本书成为你从 C 到 Rust 的桥,愿你进入生产后不再有语法困惑。